

Software Craftsmanship

La fierté

La première valeur qu'on va aller chercher c'est la fierté. Alors attention soyons clairs, on ne parle pas là d'une espèce d'arrogance ou d'une espèce de fierté mal placée.

Non je parle de quelque chose de plus profond, cette espèce de fierté du travail accompli, du travail bien fait, que tu peux ressentir à la fin de la journée quand tu as le sentiment d'avoir vraiment donné le maximum de toi même.

Quelque part la fierté c'est ce qui donne de la noblesse à ton travail et je ne connais pas grand monde qui n'ait pas tellement envie d'être fier de son travail. Voire même je constate que c'est une valeur franchement puissante pour mobiliser les gens et les motiver.

L'excellence

La deuxième valeur est pour moi celle de l'excellence. Alors attention on pourrait facilement tomber dans un piège. Pour moi l'excellence n'est pas un espèce d'idéal de perfection. L'excellence est, pour moi, une démarche permanente d'amélioration. C'est une espèce d'objectif qui ne sert jamais qu'à donner le chemin, mais ce n'est pas le chemin.

Le chemin c'est ce travail que tu as à faire au quotidien pour progresser que ce soit dans tes pratiques ou progresser sur toi dans ton interaction avec le monde et tendre vers un espèce d'idéal qui te guide.

Le pragmatisme

Enfin, dernière valeur que je te propose : le pragmatisme. C'est ce qui permet de rester ancrée, de garder bien les pieds sur terre, et qui vient équilibrer les deux premières valeurs. Parce que sans le pragmatisme tu peux facilement t'égarer dans une espèce de labyrinthe et perdre de vue ce qui est réellement important.

Ce qui est réellement important ce n'est pas forcément le code que tu es en train d'écrire mais à quoi il va servir.

Quand tu as un dilemme du type : j'ai deux options A et B. – A me fait gagner du temps, me permet d'aller vite, mais je sens que ce que je vais faire n'est pas terrible. – B prend plus de temps, mais je sens que c'est plus propre et plus durable.

Si tu n'actionnes que l'excellence et la fierté, tu vas systématiquement choisir l'option B. Or il faut savoir parfois prendre l'option A : l'exemple typique est celui d'une opportunité client ou d'un événement dont tu ne contrôle pas la date, comme une conférence. Ou alors tu peux avoir un dilemme sur la manière d'optimiser ton code.

A l'inverse, si tu n'actionnes que le pragmatisme, et que tu choisis systématiquement l'option A, tu le paieras tôt ou tard.

La transmission

La transmission des connaissances, de l'état d'esprit et des savoir-faire est un enjeu majeur de l'artisanat logiciel.

La meilleure manière d'apprendre et de transmettre reste l'échange de pair à pair. On parle de pratique longues à acquérir. Il est facile de faire fausse route et de se décourager. Sur le chemin, trouver un mentor sera une aide précieuse pour avancer.

A l'inverse, quand tu as de l'expertise à partager, devenir mentor est un excellent moyen de renforcer ses connaissances. Mais il faut le faire dans de bonnes conditions. Si cela t'intéresse, j'ai fait une section dédiée à la transmission dans cette boîte à outils du leader craft.

L'extreme programming fait du travail en binôme une pratique centrale et il y a une bonne raison à ça : c'est un moyen redoutablement efficace de transmettre et homogénéiser les connaissances en faisant monter le niveau de chaque individu.

Personnellement, j'en ferais un critère de choix majeur si je cherchais un travail : avec qui vais-je travailler ? Qu'est-ce que je vais pouvoir apprendre ou transmettre ?

Les principes du Software Craftsmanship

L'amélioration continue

Pour aller sur le chemin de l'excellence il faut être en permanence en train de s'améliorer et ça, c'est une vraie démarche d'humilité. S'améliorer en continue, ce voir qu'on peut faire mieux la prochaine fois.

Être professionnel

Etre professionnel, pour moi, ça veut dire faire des choses qui ne sont pas naturelles, qui ne sont pas forcément faciles, dans l'intérêt du projet, parce que justement on est des professionnels.

Le but c'est d'amener le projet le plus loin possible. Quelque chose par exemple qui n'est pas naturel pour plein de monde, c'est d'aller demander un feedback ou alors de donner un feedback. Et quand je parle de donner un feedback, je parle de de donner vraiment un feedback sincère. D'ailleurs j'ai fait une vidéo sur le sujet si ça t'intéresse d'aller plus loin.

Etre professionnel c'est être profondément responsable de son travail responsable, de ce qu'on fait. Etre responsable, ça peut vouloir dire par exemple poser des limites, savoir dire non. Finalement en tant que professionnel, si on se retrouve dans une situation qui nous paraît juste pas acceptable, pas possible, eh bien il faut savoir dire non.

J'aime bien cet exemple qui revient souvent, mais qui marque bien les gens et qui est assez explicite : imagine que tu ailles voir qu'un chirurgien et au moment où il va opérer tu lui dis : "Non, non, ne vous lavez pas les mains ça sert à rien. On va gagner du temps."

Là si c'est toi le chirurgien, est-ce que tu acceptes de faire ça ? Est ce que ça te paraît être une bonne idée de grignoter sur quelque chose aussi essentiel pour gagner un petit peu de temps, et peut-être le payer après ?

Les pratiques

L'artisanat logiciel, c'est aussi tout un ensemble de pratiques, que ce soit des pratiques d'équipes comme le binomage par exemple, les pratiques de conception avec le Test Driven Design ou les principes SOLID, ou

encore avec l'architecture hexagonale. Tu vas d'ailleurs trouver encore plein d'autres choses avec le Domain Driven Design.

En creusant, on peut trouver beaucoup d'acronymes qui caractérisent l'artisanal logiciel. D'ailleurs, j'ai essayé une fois de faire une diapo qui rassemble un maximum de mots clés et j'ai déjà rempli une bonne page d'acronymes sans avoir été exhaustif.

Je comprends que ça puisse être un petit peu intimidant pour ceux qui découvrent ça au début. Mais crois moi, ça vaut vraiment le coup d'aller voir ce qui se cache derrière.

D'ailleurs si tu as envie de faire un diagnostic de tes pratiques et comparer ton niveau à celui de centaines d'autres développeurs, je t'invite dans [compagnon](#) pour faire ton diagnostic de pratiques.

- en plus : **SOLID**
 - Single Responsibility
 - Open/Close principe : ouvert à l'extension fermé aux modifications
 - Liskof substitution : objets des sous classes peuvent remplacer les objets des superclasses sans affecter le soft
 - interface segregation : ne pas dépendre d'interfaces que l'on utilise pas
 - dependency inversion

Le pair programming

Le pair programming, ou travail en binôme, est une pratique qui consiste à travailler à deux, en même temps, sur une même tâche de développement.

Cette pratique formalisée dans l'eXtreme Programming est peut-être encore une des plus controversée du mouvement. Souvent incomprise ou mal employée, c'est pourtant un outil d'une grande puissance quand on sait s'en servir. C'est pour ça que je t'ai écrit le guide ultime du pair programming. Oui, je sais, le titre est un peu prétentieux. Mais en vrai, il y a peu de gens qui ont une expérience profonde de ce mode de travail, et encore moins qui soient capables d'en parler. Alors je me suis dit que ça valait le coup de s'y coller.

Le mob programming

Le mob, c'est comme le pair, mais à plusieurs. Si le pair est souvent incompris, je crois que le mob l'est encore plus.

C'est pour ça qu'on a t'a préparé un guide aux petits oignons sur le mob programming.

Le refactoring

Le refactoring de code, c'est l'art de retravailler et d'améliorer la structure interne du code, sans en changer le comportement extérieur. C'est un peu comme ranger et organiser ton atelier après une grosse session de bricolage. Pourquoi c'est important ? Eh bien, cela rend ton code plus lisible, plus maintenable et plus évolutif. En gros, c'est comme huiler régulièrement les rouages de ta machine pour qu'elle tourne comme une horloge.

Quand tu es rodé, c'est quelque chose que tu fais en continue, mais si ce n'est pas dans tes habitudes et que tu te demandes s'il est temps d'un bon coup de nettoyage, voici 5 signes qui indiquent qu'il est temps d'un bon refactoring.